AUTOMAÇÃO WEB DETERMINÍSTICA VS. AGÊNTICA: UMA AVALIAÇÃO EMPÍRICA DE CONFIABILIDADE, CUSTO E ROBUSTEZ

DETERMINISTIC VS. AGENTIC WEB AUTOMATION: AN EMPIRICAL EVALUATION OF RELIABILITY, COST AND ROBUSTNESS

Alcalde, Luca Jordão Salas[a]. Santiago, Fábio[b]
[a]Centro Universitário Facens — Sorocaba, SP, Brasil
lucajsalcalde@gmail.com
[b]Centro Universitário Facens — Sorocaba, SP, Brasil
fabio.santiago@facens.br

Submetido em: 24 out. de 2026. Aceito em: ___ de _____ de 2026.

RESUMO

A automação web determinística encontra os elementos da página por endereços fixos, os localizadores, e quebra quando a página muda; agentes de modelos de linguagem prometem adaptar-se. A literatura mede essa quebra em mudanças reais, que misturam tipo e tamanho, e avalia agentes sem compará-los à automação determinística. Este trabalho as compara, no contexto da automação robótica de processos, alterando a página de forma controlada (degradação): cinco abordagens (ID/CSS, XPath, papel e rótulo, *self-healing*, que procura um substituto quando o localizador falha, e agêntica), três tipos cumulativos de degradação, intensidades de 0 a 1 e duas aplicações de teste, em 1.200 execuções. Cada determinística caiu sob a degradação que altera a informação em que se apoia: a ID/CSS concluiu 10,9% das execuções sob a cosmética, a XPath, 6,2% sob a estrutural, e papel e rótulo e o *self-healing*, 26,6% e 89,1% sob a semântica. A agêntica nunca falhou. Sob degradação semântica de 0,50 ou mais, concluiu 48 de 48 execuções, contra 41 do *self-healing* (McNemar exato, p = 0,016), diferença concentrada numa aplicação. Cada execução agêntica custou, em média, US$ 0,119 e levou, em mediana, 36,7 s, contra 8,6 s do *self-healing*, e só compensa sob mudança semântica frequente e baixo volume. Como as degradações preservam o significado lido pelo agente, o experimento não mostra até onde ele resiste. Para organizações que mantêm robôs, o trabalho oferece um mapa de fragilidade por tipo de mudança, um critério de custo e um instrumento aberto.

Palavras-chave: Automação web. Agentes de modelos de linguagem. Robustez de software. Degradação de interface. Experimento controlado.

ABSTRACT

Deterministic web automation finds page elements through fixed addresses, the locators, and breaks when the page changes; language model agents promise to adapt. The literature measures this breakage under real changes, where type and size come mixed, and evaluates agents without comparing them to deterministic automation. This work compares them, in the setting of robotic process automation, by changing the page in a controlled way (degradation): five approaches (ID/CSS, XPath, role and label, self-healing, which looks for a substitute when a locator fails, and agentic), three cumulative degradation types, intensities from 0 to 1 and two test applications, in 1,200 runs. Each deterministic approach fell under the degradation that alters the information it relies on: ID/CSS completed 10.9% of runs under cosmetic degradation, XPath 6.2% under structural, and role and label and self-healing 26.6% and 89.1% under semantic. The agentic approach never failed. Under semantic degradation of 0.50 or more, it completed 48 of 48 runs, against 41 for self-healing (exact McNemar, p = 0.016), a difference concentrated in one application. Each agentic run cost a mean of US$ 0.119 and took a median of 36.7 s, against 8.6 s for self-healing, and it only pays off under frequent semantic change and low volume. Since the degradations preserve the meaning the agent reads, the experiment does not show how far it holds. For organizations that maintain bots, the work offers a fragility map by type of change, a cost criterion and an open instrument.

Keywords: Web automation. Language model agents. Software robustness. Interface degradation. Controlled experiment.

1 INTRODUÇÃO

Na automação robótica de processos (RPA, do inglês *robotic process automation*), programas operam sistemas de terceiros pela tela usada por pessoas, sem acesso ao código desses sistemas e sem garantia de que a tela continue igual (VAN DER AALST; BICHLER; HEINZL, 2018). É o caso da automação de obrigações fiscais em portais públicos. Essa automação é determinística: o programa encontra cada campo e cada botão por um endereço fixo, chamado localizador, e executa ações programadas, o que a torna previsível e barata, mas a faz quebrar quando a página muda. Já os agentes baseados em modelos de linguagem recebem a tarefa em linguagem natural e decidem sozinhos onde clicar, sem localizadores definidos de antemão, e por isso prometem adaptar-se a essas mudanças. Em troca, cobram por execução, demoram mais e não repetem sempre o mesmo comportamento.

A quebra dos localizadores é bem documentada na engenharia de software, mas foi medida em mudanças reais das aplicações, em que o tipo e o tamanho da mudança vêm misturados e não podem ser separados (HAMMOUDI; ROTHERMEL; TONELLA, 2016; LEOTTA et al., 2016). Os testes de robustez de agentes injetam falhas controladas no ambiente, mas avaliam só agentes, sem incluir a automação determinística (KARA; FAISAL; NATH, 2026). E o único trabalho encontrado que compara automação determinística e agêntica não trata o tipo nem a intensidade da mudança como variável (PRŮCHA; MATOUŠKOVÁ; STRNAD, 2026).

Diante disso, este trabalho pergunta sob quais tipos e intensidades de mudança na página a automação agêntica conclui tarefas com mais frequência que a determinística, e a que custo. Para responder, construiu-se um instrumento que altera a página de forma controlada e repetível, sem mudar o funcionamento das aplicações: a degradação, de três tipos (seção 3.4) e intensidade de 0 a 1. Com ele mediu-se a taxa de sucesso de cinco abordagens: ID/CSS, XPath, papel e rótulo, *self-healing* e agêntica (seção 3.5). A hipótese principal, registrada antes da coleta, é que, sob degradação semântica de intensidade igual ou superior a 0,50, a taxa de sucesso da agêntica difere da do *self-healing*, a determinística que procura sozinha um substituto quando o localizador falha e que foi a mais resistente numa coleta preliminar (seção 3.7). Além dela, estimam-se a queda do sucesso com a intensidade, o acerto das substituições do *self-healing*, a diferença entre as aplicações, o tempo e o custo anual.

Para organizações que mantêm automações de processos, a contribuição prática é dar três apoios à decisão: um mapa de quais mudanças de página derrubam cada forma de localizar elementos, para escolher o localizador pelo tipo de mudança mais comum nos sistemas automatizados; um modelo de custo que indica a partir de quantas mudanças por ano, e com que volume de execuções, um agente compensa o seu preço; e um instrumento aberto, adaptável às páginas da própria organização, para testar uma automação antes de pô-la em produção.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Conceitos

No navegador, a página é representada como uma árvore de elementos, o modelo de objetos do documento (DOM), que os programas podem consultar e alterar. Localizador é a expressão que aponta um elemento nessa árvore, e as formas mais comuns são três: pelo identificador (*id*) ou pelas classes de estilo (CSS), nomes dados pelo programador; pelo caminho na árvore, escrito na linguagem XPath, que aqui parte sempre da raiz; e pelo papel e pelo rótulo do elemento, como "botão Login", na árvore de acessibilidade, a versão simplificada da página usada por leitores de tela. *Self-healing* é o mecanismo que, quando um localizador falha, procura na página o elemento mais parecido com o original; cada substituição é chamada aqui de cura.

Um modelo de linguagem é um programa treinado em grandes volumes de texto para gerar texto a partir de instruções. Um agente é um programa em que o modelo decide, a cada passo, que ações executar, como clicar ou digitar. O Model Context Protocol (MCP) é o padrão que liga o modelo a essas ações, e o servidor Playwright MCP, baseado no Playwright, biblioteca que controla o navegador, entrega a página ao agente na forma da árvore de acessibilidade. Oráculo é a verificação que decide se a tarefa foi concluída pelo estado da aplicação, e não pelo relato da automação.

2.2 Fragilidade de localizadores

Os estudos a seguir são de testes automatizados, mas a RPA usa as mesmas ferramentas e localizadores e quebra da mesma forma. Hammoudi, Rothermel e Tonella (2016) atribuíram ao localizador mais de 73% das quebras de testes em oito aplicações acompanhadas ao longo de suas versões. Leotta et al. (2016) viram 871 de 1.110 localizadores XPath absolutos, que partem da raiz, deixarem de funcionar na versão seguinte das aplicações. Antes, Leotta et al. (2014) viram a remoção de um

único contêiner sem uso, um bloco que só agrupa outros elementos, quebrar 36 localizadores XPath sem alterar a aparência da página. Nass et al. (2023) propuseram o Similo, que compara atributos do elemento procurado com os da página e escolhe o mais parecido, o mesmo princípio do *self-healing* deste trabalho.

2.3 Conserto automático e *self-healing*

O conserto automático de localizadores, que corrige o teste depois da quebra, e não durante a execução, como o *self-healing*, acerta até 81% das quebras ao reconhecer o elemento pela imagem da tela (STOCCO; YANDRAPALLY; MESBAH, 2018), mas tem um risco: devolver o elemento errado. Os mesmos autores mostraram que uma técnica baseada nos atributos da página não percebeu nenhum dos 38 casos desse tipo, porque o elemento errado estava entre os candidatos que ela mesma considerava válidos. Qi, Qian e Li (2023) reduzem esse risco submetendo cada conserto a uma pessoa; este trabalho confere cada cura do *self-healing* contra a identidade original do elemento (seção 3.4).

2.4 Arquiteturas de agentes e soluções comerciais

Jin et al. (2025) agrupam as arquiteturas de automação em quatro paradigmas: sistemas baseados em regras, como a RPA tradicional, fáceis de auditar, mas de manutenção manual; sistemas baseados em modelos de linguagem; sistemas multiagentes, em que agentes coordenados dividem a tarefa; e sistemas híbridos, que combinam regras e inteligência artificial, como o UiPath AI Fabric. Aqui, as determinísticas pertencem ao primeiro paradigma, e a agêntica, um único agente, ao segundo. Os agentes também diferem no modo de ler a página: alguns, como o deste trabalho, recebem a árvore de acessibilidade, em texto; outros recebem capturas de tela e clicam por coordenadas, como os agentes comerciais Operator, da OpenAI, e Claude Computer Use, da Anthropic (XUE et al., 2025), e por isso veriam os deslocamentos da degradação estrutural (seção 3.4).

Do lado comercial, a RPA é vendida por plataformas como o UiPath; González Enríquez et al. (2020) compararam 14 delas e viram que a maioria não ajuda a escolher os processos a automatizar. Os agentes comerciais disputam esse espaço, mas ainda falham muito: em tarefas reais na web, o Operator concluiu 61% e o Claude Computer Use, até 56%, contra cerca de 30% dos agentes de código aberto, como SeeAct e Browser Use (XUE et al., 2025).

O WAREX, conjunto de testes de robustez, injeta erros de rede e de servidor e janelas sobrepostas em tarefas de agentes e derruba o sucesso deles (KARA;

FAISAL; NATH, 2026). Xue et al. (2025) mostraram que, quando um modelo de linguagem julga se o agente concluiu a tarefa, respostas inventadas pelo agente produzem falsos sucessos; por isso o oráculo deste trabalho confere o estado da aplicação (seção 3.6). A comparação direta mais próxima, de Průcha, Matoušková e Strnad (2026), confrontou o UiPath e o Claude Computer Use em 51 execuções: a RPA foi mais rápida, sem diferença significativa de confiabilidade, e a vantagem atribuída ao agente em telas que mudam vem de uma única execução dele, num formulário que muda de leiaute, interrompida antes do fim.

3 METODOLOGIA

Esta pesquisa é experimental e quantitativa. Ao longo da seção, chama-se sinal cada informação que permite identificar um elemento: o identificador, as classes, a posição na árvore, o papel, o rótulo e o texto.

3.1 Busca da literatura

A busca foi feita em 21 de agosto de 2026 em seis bases (arXiv, ACM Digital Library, IEEE Xplore, SpringerLink, Wiley Online Library e Google Scholar), de 2014 a 2026, em inglês e português, com quatro consultas, e incluiu, pela leitura de títulos e resumos, os trabalhos empíricos sobre robustez de automação web, formas de localização ou agentes em páginas web. É uma busca estruturada, e não uma revisão sistemática: dos doze trabalhos achados, mais dois das suas referências, saíram quatro sem relação direta com a pergunta e três *preprints*, trabalhos divulgados sem a revisão de outros pesquisadores, o que deixa sete; de um deles, também *preprint*, usa-se a versão depois publicada em revista (PRŮCHA; MATOUŠKOVÁ; STRNAD, 2026). Na Biblioteca Digital Brasileira de Teses e Dissertações (BDTD), na SciELO e no Portal de Periódicos da CAPES, em 7 de setembro de 2026, não se achou trabalho nacional sobre o tema. Os demais trabalhos citados entraram fora desse protocolo; consultas e levantamento estão no repositório.

3.2 Desenho experimental

O experimento cruza a abordagem (cinco), o tipo de degradação (três), a intensidade (cinco: 0; 0,25; 0,50; 0,75 e 1,0) e a aplicação (duas), com oito repetições: 5 × 3 × 5 × 2 × 8 = 1.200 execuções. As 240 com intensidade 0, sem alteração, formam a condição de controle e medem a confiabilidade: se a abordagem conclui a tarefa quando nada muda, sem falso sucesso (seção 3.6); como sem alteração o tipo não faz diferença, são, na prática, três repetições da mesma condição. As outras 960 medem a robustez,

quanto desse sucesso se mantém quando a página muda.

As repetições se distinguem pela semente, um número que fixa quais elementos da página são sorteados para alteração: com a mesma semente, o sorteio se repete. Na repetição 3 com degradação semântica de 0,50, por exemplo, as cinco abordagens recebem a página alterada pelo mesmo sorteio, e os resultados podem ser comparados em pares. A página, porém, nem sempre sai idêntica, porque o sorteio segue a ordem dos elementos no momento da degradação: se a página tiver nesse momento um elemento a mais ou a menos, o sorteio pode cair em outro. A hipótese principal usa só a degradação semântica de 0,50 a 1,0, o mesmo recorte da coleta preliminar, o que dá 48 pares de execuções da agêntica e do *self-healing* (2 aplicações × 3 intensidades × 8 sementes). Pelo teste usado (seção 3.8), a diferença só pode ser significativa com pelo menos seis pares em que só uma das duas concluiu.

3.3 Ambiente e tarefas

Adotaram-se duas aplicações públicas, mantidas por terceiros para a prática de automação de testes, fora do controle do autor, como os sistemas que a RPA opera. No SauceDemo (www.saucedemo.com), loja virtual que troca de página sem recarregá-la, a tarefa faz *login*, põe um produto no carrinho e finaliza o pedido, em cinco páginas; no The Internet (the-internet.herokuapp.com), site de páginas simples, é preciso fazer *login*, adicionar três elementos que surgem ao clicar num botão e remover um. A degradação é feita por um programa em JavaScript que o navegador executa a cada página aberta, inclusive nas trocas sem recarga do SauceDemo: ele altera a página já montada, sem modificar a aplicação no servidor. As determinísticas usam o Playwright 1.61 para Python e esperam até 10 s por elemento.

3.4 Degradação da interface

Os três tipos de degradação são cumulativos e inspiram-se em mudanças comuns: a troca da biblioteca de estilos, o redesenho do leiaute e a revisão dos textos. A cosmética renomeia identificadores e classes: o *id* "login-button" vira "login-button-48213". A estrutural acrescenta três operações: envolve elementos em contêineres que não aparecem na tela, o inverso da remoção de contêiner que quebrou XPaths em Leotta et al. (2014), desloca campos e botões em até 40 pixels e insere cópias ocultas, de modo que o XPath do campo de usuário passa a apontar para a cópia. A semântica acrescenta ainda um sufixo a textos e rótulos: o botão "Login" vira "Login_807".

A intensidade é a probabilidade de cada elemento ser alterado; nas operações estruturais, usa-se uma fração dela, com os valores no repositório. Só os

identificadores de campos, botões e links são renomeados, porque os demais são usados pelo próprio código da aplicação; o *name* e o *value* dos campos ficam intactos, porque são o que o formulário envia ao servidor; e os atributos de teste, como *data-test*, também são preservados. As intensidades são aninhadas: em regra, com a mesma semente, o que se altera em 0,25 continua alterado nas intensidades maiores. Cada elemento alterado guarda a identidade original, usada só para conferir resultados.

3.5 Abordagens comparadas

As cinco abordagens usam o mesmo navegador, o Chromium, e diferem apenas no sinal em que se apoiam para encontrar os elementos. A ID/CSS usa o identificador e as classes, e deve cair sob a degradação cosmética. A XPath segue o caminho completo desde a raiz da página, e deve cair sob a estrutural. As duas seguintes apoiam-se no que está escrito na tela e são vulneráveis à semântica: papel e rótulo procura, por exemplo, o botão de rótulo exato "Login", e o *self-healing* compara vários atributos do elemento. A agêntica lê a página pela árvore de acessibilidade, que traz o papel e o rótulo dos elementos, mas não os identificadores, as classes ou a posição na tela, nem os blocos e as cópias ocultas que a degradação estrutural acrescenta (seção 3.4); por isso, só a semântica pode atingi-la. Nenhuma determinística usa atributos criados só para testes, como *data-test*, porque sistemas de terceiros não os garantem. Papel e rótulo exige o rótulo exato: no modo padrão, o Playwright aceita rótulos que só contenham o texto procurado (MICROSOFT, [2026]), e o sufixo nunca a quebraria, ao contrário de uma troca de palavra, como "Login" por "Entrar".

O *self-healing* usa os mesmos localizadores da ID/CSS e, por isso, é acionado já sob a degradação cosmética. Quando um deles falha, ele compara os elementos da página com um perfil do elemento procurado, gravado numa execução sem degradação, e dá a cada um uma nota de semelhança de 0 a 1. O melhor é aceito se a nota for alta (limiar de 0,62) e claramente maior que a do segundo (margem de 0,10); senão, o mecanismo desiste e a execução falha. Antes de comparar, espera até 3 s pelo localizador original, para não agir antes de a página carregar. O perfil reúne tipo, atributos (como *name*; *placeholder*, a dica dentro do campo vazio; e *aria-label*, o rótulo lido por leitores de tela), texto, classes e posição, com pesos de 0,10, 0,35, 0,25, 0,10 e 0,20, redistribuídos quando falta informação, como o texto de um campo. Só concorrem elementos do mesmo tipo (campo com campo, botão com botão), e por

isso a parcela do tipo, igual para todos, só pesa na comparação com o limiar. Pesos, limiar e margem foram escolhidos pelo autor, sem base em outro trabalho, e fixados antes da coleta. Conferidos antes da coleta nas telas de *login*, com outras sementes (300 tentativas de cura: cinco campos e botões, três tipos, quatro intensidades e cinco sementes), qualquer limiar até 0,88 e margem de 0 a 0,15 acertaram todas; o formulário de finalização do SauceDemo ficou fora dessa conferência.

A agêntica usa o modelo claude-sonnet-5, a biblioteca claude-agent-sdk 0.2.128, que conduz o ciclo do agente, e o Playwright MCP 0.0.81, no mesmo navegador degradado. O agente recebe a tarefa em linguagem natural, sem localizadores, e a instrução de examinar a página antes de agir (textos no repositório). Cada rodada de resposta do modelo, em que ele escolhe uma ou mais ações, como clicar ou ler a página, conta como um passo. Ele só dispõe das ações do navegador, sem a de executar código na página; a de clicar aceita tanto um elemento lido na árvore quanto um localizador escrito pelo próprio agente. A execução com degradação em que o agente usa *data-test*, *data-testid* ou *data-qa* é registrada como contorno da interface, porque indica um atalho, e não robustez. Para conter o gasto se o agente entrar em repetição, o autor fixou dois limites: 40 passos, com folga sobre os 18 a 20 de um piloto sem degradação, e US$ 0,60, pela estimativa de custo que a própria biblioteca calcula com uma tabela de preços (ANTHROPIC, [2026b]).

3.6 Verificação do resultado

O sucesso nunca é declarado pela automação: as verificações ficam num módulo à parte, o oráculo. No The Internet, a execução precisa entrar na área restrita e terminar com dois dos três elementos adicionados; no SauceDemo, que aceita finalizar a compra até com o carrinho vazio, chegar à confirmação com exatamente um item no carrinho. Nas determinísticas, a tarefa chama o oráculo ao fim de cada etapa, e ele confere a página sem usar o localizador da abordagem. Na agêntica, sem como intervir entre as ações do agente, o oráculo instala na página programas que registram o que as determinísticas conferem a cada etapa e os lê ao final: os critérios são iguais, só muda o momento da leitura.

3.7 Coleta e validação do instrumento

Uma coleta preliminar, em agosto de 2026, foi descartada por três defeitos do instrumento, descritos no plano de análise, entre eles um oráculo menos exigente para a agêntica. Dela restaram apenas o número de repetições, sem cálculo prévio do tamanho da amostra, o recorte de 0,50 a 1,0 da hipótese e a escolha do *self-healing*

como comparador, que continuou a determinística mais resistente na coleta definitiva.

A coleta definitiva ocorreu de 17 a 20 de setembro de 2026, depois de duas correções do instrumento registradas no plano de análise: a nova captura dos caminhos XPath e dos perfis do SauceDemo, que mudara, e a espera de 3 s do *self-healing* (seção 3.5), que procurava o elemento antes de a página carregar e levou a descartar uma primeira tentativa de 66 execuções. O plano foi gravado no controle de versões local antes de qualquer execução da coleta definitiva, inclusive da tentativa descartada (versão be6bd2c, de 16 de setembro de 2026), e as correções entraram como revisões datadas, sem mudar a hipótese; plano, código, dados e registros do agente estão em https://github.com/lucajsalcalde/automacao-web-deterministica-vs-agentica. Antes da coleta, as determinísticas só falharam onde se previa, e um programa que localiza os elementos pela identidade original concluiu as tarefas nas 192 combinações degradadas. Por limite de uso do modelo, a coleta teve dois blocos, determinísticas e agêntica, cada um em ordem sorteada, e a verificação das determinísticas, repetida entre eles, não mudou. Das 89 condições agênticas executadas duas vezes por erro de operação, manteve-se a última, sem divergência, e vinte verificações automáticas dos dados não acusaram erro.

O custo de cada execução agêntica foi calculado pelos *tokens*, as unidades de texto que o modelo lê e gera, ao preço público do modelo (ANTHROPIC, [2026a]). A estimativa da biblioteca (seção 3.5), feita com tabela de preços própria, é maior, US$ 43,19 no total; ao preço público, o limite de US$ 0,60 equivale a cerca de US$ 0,40.

3.8 Métodos de análise

O Quadro 1 liga cada pergunta ao teste que a responde. O intervalo de confiança (IC) de 95% dá a margem de incerteza de cada taxa: com 16 execuções, 100% de sucesso observado é compatível com uma taxa real de 81% a 100% (intervalo de Wilson). O valor de p é a probabilidade de uma diferença tão grande surgir só por acaso se as abordagens fossem iguais (exemplo na seção 4.2); abaixo de 0,05, a diferença é considerada real. Quando o mesmo teste se repete em muitas condições, a correção de Holm torna esse critério mais rigoroso. Só a hipótese principal, que exclui as execuções de contorno, foi registrada antes da coleta; as demais análises são exploratórias.

Quadro 1 – Perguntas da análise e procedimentos estatísticos

Pergunta	**Procedimento e o que ele faz**
A agêntica conclui mais	Teste exato de McNemar: compara as duas só nos pares em

Pergunta	Procedimento e o que ele faz
que o *self-healing* sob degradação semântica de 0,50 a 1,0? (hipótese principal)	que uma concluiu e a outra não (seção 4.2), no total e em cada aplicação, com o IC de Agresti e Min (2005) para a diferença.
Quais abordagens diferem? O resultado depende da aplicação?	O mesmo teste, nas 120 comparações entre abordagens, com a correção de Holm; teste exato de Fisher entre as aplicações.
O sucesso cai quando a intensidade aumenta?	Teste de Cochran-Armitage, que procura uma queda ao longo dos níveis de intensidade.
A agêntica é mais lenta? O custo muda com a intensidade?	Testes de Mann-Whitney, para o tempo, e de Kruskal-Wallis, para o custo.

Fonte: elaborado pelo autor (2026).

3.9 Modelo de custo anual

O custo anual segue o modelo de ponto de equilíbrio de Leotta et al. (2013), que somaram o custo de manutenção de duas formas de automação de testes a cada versão da aplicação; aqui a versão dá lugar à mudança de interface, e soma-se o custo por execução, só da agêntica, relatado em dólares pelos *tokens* (KAPOOR et al., 2025):

$$C = M \cdot f \cdot c_{int} + E \cdot c_{exec} \qquad (1)$$

em que M é o número de mudanças de interface por ano; f, a proporção de falhas da abordagem sob o tipo de mudança considerado, medida em todas as intensidades com degradação, porque uma mudança real pode ter qualquer tamanho; c_{int}, o custo de uma intervenção; E, o número de execuções por ano; e c_{exec}, o custo médio de uma execução concluída com degradação (a média, porque o custo anual é uma soma), nulo nas determinísticas. Supõe-se que cada mudança real se comporte como uma degradação sorteada do mesmo tipo e, com probabilidade f, exija um conserto. Igualando as expressões de duas abordagens, obtém-se o ponto de empate, número de mudanças por ano a partir do qual a agêntica fica mais barata. Como este experimento não mede consertos, c_{int} é premissa do autor, sem fonte: meia hora de trabalho a R$ 250,00 por hora, ou R$ 125,00 por conserto, com o valor da hora variado de R$ 100,00 a R$ 300,00. O câmbio, também premissa, é de R$ 5,19 por dólar, cotação de 20 de agosto de 2026 (±10%), e os volumes, de uma, vinte e duzentas execuções por dia útil, em 252 dias úteis por ano, sem os custos de construção e de máquina.

4 RESULTADOS

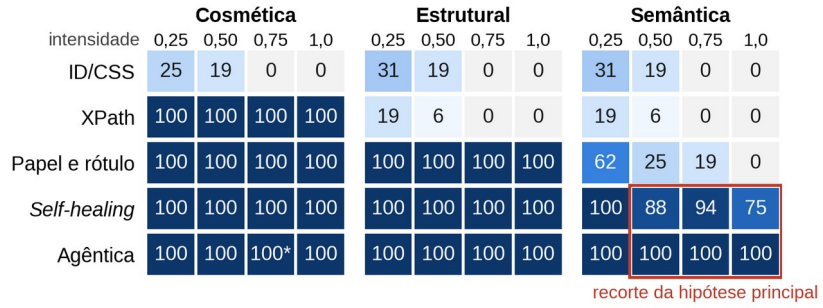
A condição de controle, sem degradação, teve 240 sucessos em 240 execuções: com

a página íntegra, as cinco abordagens foram confiáveis. Não houve falso sucesso: todas as falhas foram interrupções da própria automação, por elemento não encontrado ou desistência do *self-healing*.

4.1 Robustez por tipo de degradação

A Figura 1, aqui chamada de mapa de fragilidade, se lê por linha. Cada determinística mantém 100% até o tipo de degradação que altera o sinal em que se apoia e cai a partir dele, também nos tipos seguintes, que o incluem. A queda é rápida porque basta um dos vários elementos da tarefa alterado para a execução falhar. Diferenças pequenas entre tipos, como os 25% e 31% da ID/CSS em 0,25, devem vir da variação da página degradada (seção 3.2). Somadas as quatro intensidades, a ID/CSS ficou em 10,9% sob a cosmética, a XPath em 6,2% sob a estrutural, e papel e rótulo e o *self-healing* em 26,6% e 89,1% sob a semântica. A agêntica não falhou em nenhuma condição: concluiu 239 das 240 execuções, e a restante, sob degradação cosmética de 0,75, chegou ao fim pelo atributo *data-test* e saiu das taxas como contorno. Como atributos de teste não aparecem na árvore de acessibilidade, o localizador [data-test="checkout"], escrito pelo agente, indica que o modelo já conhecia o SauceDemo, provavelmente do seu treinamento.

Figura 1 – Taxa de sucesso (%) por abordagem, tipo e intensidade de degradação



Fonte: elaborado pelo autor (2026). 16 execuções por célula, 8 por aplicação. *15 execuções (um contorno excluído). Arredondamento com ,5 ao par (62,5 vira 62); intensidade 0, com 100% em todas, omitida; aplicações somadas (a vantagem da agêntica vem do SauceDemo, seção 4.2).

Nas sete combinações de abordagem e tipo com falhas, o sucesso tendeu a cair com a intensidade (Cochran-Armitage). Das 120 comparações entre abordagens, 55 mostraram diferença real, sempre contra a abordagem atingida.

4.2 Hipótese principal

A hipótese principal compara os 48 pares das células destacadas na Figura 1 (seção 3.2). A agêntica concluiu os 48, e o *self-healing*, 41, as células de 88%, 94% e 75%. Em 41 pares as duas concluíram, em 7 só a agêntica e em nenhum só o *self-healing*. O teste de McNemar olha só esses 7 pares, em que as duas divergiram. Se as

abordagens fossem igualmente boas, cada um deles poderia favorecer qualquer uma, como no cara ou coroa, e a probabilidade de os 7 caírem todos do mesmo lado, para qualquer das duas, seria $2 \times 0{,}5^7 = 0{,}016$. Como 0,016 é menor que 0,05, a diferença é considerada real: 14,6 pontos percentuais a favor da agêntica (IC 95% de 3,6 a 24,4). Célula a célula, nenhuma das 12 comparações entre as duas foi significativa, porque nenhuma célula teve mais de quatro pares divergentes, menos que os seis necessários (seção 3.2); a diferença só aparece ao juntar as três intensidades.

A diferença vem toda do SauceDemo, com 24 de 24 contra 17 de 24 ($p = 0{,}016$; diferença de 29,2 pontos, IC 95% de 8,3 a 45,6); no The Internet, as duas concluíram 24 de 24. Pelo teste de Fisher, só o *self-healing* mudou entre as aplicações ($p = 0{,}047$, já com a correção de Holm). Nenhum par teve contorno.

4.3 Curas do *self-healing*, tempo e custo

As falhas do *self-healing* foram desistências, não curas erradas: ele foi acionado 554 vezes, curou 547, todas no elemento certo, e desistiu 7, nos campos de nome e sobrenome da finalização do SauceDemo, sem a margem de 0,10 sobre o segundo.

A execução agêntica concluída levou, em mediana, 36,7 s, contra 8,6 s do *self-healing* e cerca de 2,3 s das outras (Mann-Whitney, $p < 0{,}001$ contra cada uma). No SauceDemo, o agente usou de 16 a 28 passos na mesma tarefa. O tempo do *self-healing* vem da espera de 3 s por acionamento (seção 3.5): sem cura, levou 2,4 s; com cura, 8,8 s, ou 2,2 s sem a espera. Ao preço público, as 191 execuções agênticas concluídas com degradação custaram em média US$ 0,119 (mediana de US$ 0,112); as 240, US$ 28,65. O custo não variou de forma detectável com a intensidade (Kruskal-Wallis, $p = 0{,}79$), o que justifica um único c_{exec}.

Sob mudanças cosméticas e estruturais, a agêntica nunca compensa, porque há sempre uma determinística sem falhas. Sob mudanças semânticas, o *self-healing* não custa nada por execução, mas falha em 10,9% das mudanças (7 de 64), e cada uma lhe custa, em média, $7/64 \times$ R$ 125,00 = R$ 13,67 de conserto; a agêntica não precisou de conserto, mas custa R$ 0,62 por execução (US$ 0,119 × R$ 5,19). Com uma execução por dia útil, ela custa cerca de R$ 156 por ano, o mesmo que o *self-healing* gasta com 11,4 mudanças semânticas: abaixo disso, o *self-healing* é mais barato; acima, a agêntica. Com vinte execuções por dia, o empate sobe para cerca de 228 mudanças por ano; com duzentas, o *self-healing* vence mesmo que a página mude todo dia útil. Nas faixas de hora e câmbio, esse empate vai de 8,6 a 31,4 mudanças; com o custo mediano, fica em 10,7.

5 DISCUSSÃO

Como Hammoudi, Rothermel e Tonella (2016) e Leotta et al. (2016) observaram em mudanças reais, a fragilidade depende do sinal em que cada localizador se apoia. A agêntica não falhou, o que pede cautela. Como previsto (seção 3.5), a cosmética e a estrutural não mudam o que o agente vê, e a semântica preserva a palavra: "Login_807" segue lido como "Login". Elas não levaram o agente ao limite, o que não quer dizer que ele não o tenha (seção 2.4).

Contra o *self-healing*, a vantagem é real, mas estreita: sete pares de 48, num formulário fora da conferência dos parâmetros (seção 3.5). Os dois campos empatam mesmo com o *name* intacto: têm o mesmo tipo e as mesmas classes, ficam um sob o outro e têm *name* parecidos ("firstName" e "lastName"), e o sufixo no *placeholder* e no *aria-label* bastou para aproximar as notas. Com margem de 0,05, as sete desistências (diferenças de 0,067 a 0,097) teriam virado curas, mas os registros não dizem se o candidato seria o certo: uma margem menor troca a falha visível pelo risco de cura errada. Como o modelo conhecia o SauceDemo (seção 4.1), conferiram-se os sete pares: o agente preencheu nome e sobrenome pelos elementos lidos na página, e o único rótulo que citou, no *login*, já era o alterado, "Username_82", sinal de que leu a página em vez de recorrer ao que sabia. Os 16 a 28 passos na mesma tarefa (seção 4.3) confirmam que o agente não repete o caminho. Ao contrário de Průcha, Matoušková e Strnad (2026), a vantagem vem de execuções repetidas e pareadas, e as curas, conferidas, não erraram o elemento, risco apontado por Stocco, Yandrapally e Mesbah (2018).

As limitações vêm do desenho: duas aplicações públicas, com uma tarefa curta cada, e o modelo já conhecia aquela de onde vem a vantagem do agente, a principal ameaça ao resultado; os tipos cumulativos não isolam o efeito de cada sinal; e a mesma semente não garante página idêntica, sem registro de quantas vezes (seção 3.2). A degradação artificial troca realismo por controle: não se sabe onde uma mudança real cai na escala, e o sufixo é mais brando que uma reescrita com sinônimos ou tradução. No custo, o tempo de conserto é premissa, a falha nula da agêntica (f = 0) é otimista, e o robô parado não entra na conta. O *self-healing* foi implementado pelo autor, com pesos próprios, e não é ferramenta de mercado; os pesos do Similo não foram testados, e outra implementação poderia não desistir nos mesmos campos. Os blocos da coleta correram em dias diferentes, risco atenuado pela verificação repetida; houve um só modelo, e a busca não é exaustiva.

6 CONSIDERAÇÕES FINAIS

Respondendo à pergunta inicial: sob mudanças cosméticas e estruturais, a agêntica não supera a melhor determinística, que também não falhou; sob mudanças semânticas de 0,50 ou mais, conclui mais, por diferença pequena, concentrada numa aplicação e sensível ao critério do *self-healing*, e só se paga a partir de cerca de 11 mudanças semânticas por ano com uma execução por dia útil.

O resultado mais sólido é o mapa de fragilidade: cada determinística caiu sob a degradação que atinge o seu sinal, inclusive o *self-healing*, cujas 547 curas acertaram todas. Em portais fiscais, sugere escolher o localizador pelo tipo de mudança mais comum e reservar o agente a mudanças de texto frequentes e de baixo volume; o instrumento aberto testa a escolha antes.

Ficam para trabalhos futuros degradações que derrubem o agente, como rótulos trocados por sinônimos ou traduzidos, a replicação em aplicações comerciais, com outros modelos e com agentes que leem a tela, e a medida do tempo de conserto.

REFERÊNCIAS

AGRESTI, A.; MIN, Y. Simple improved confidence intervals for comparing matched proportions. **Statistics in Medicine**, [s. l.], v. 24, n. 5, p. 729-740, mar. 2005. DOI: 10.1002/sim.1781.

ANTHROPIC. **Pricing**. [S. l.], [2026a]. Disponível em: https://platform.claude.com/docs/en/about-claude/pricing. Acesso em: 16 set. 2026.

ANTHROPIC. **Track cost and usage**: Claude Agent SDK. [S. l.], [2026b]. Disponível em: https://code.claude.com/docs/en/agent-sdk/cost-tracking. Acesso em: 22 out. 2026.

GONZÁLEZ ENRÍQUEZ, J. et al. Robotic process automation: a scientific and industrial systematic mapping study. **IEEE Access**, [s. l.], v. 8, p. 39113-39129, 2020. DOI: 10.1109/ACCESS.2020.2974934.

HAMMOUDI, M.; ROTHERMEL, G.; TONELLA, P. Why do record/replay tests of web applications break? In: IEEE INTERNATIONAL CONFERENCE ON SOFTWARE TESTING, VERIFICATION AND VALIDATION, 9., 2016, Chicago. **Anais** [...]. Piscataway: IEEE, 2016. p. 180-190. DOI: 10.1109/ICST.2016.16.

JIN, W. et al. A review of AI-driven automation technologies: latest taxonomies, existing challenges, and future prospects. **Computers, Materials & Continua**, [s. l.], v. 84, n. 3, p. 3961-4018, 2025. DOI: 10.32604/cmc.2025.067857.

KAPOOR, S. et al. AI agents that matter. **Transactions on Machine Learning Research**, [s. l.], 2025. Disponível em: https://openreview.net/forum?id=Zy4uFzMviZ. Acesso em: 22 out. 2026.

KARA, S.; FAISAL, F. E.; NATH, S. WAREX: web agent reliability evaluation on existing benchmarks. **Transactions on Machine Learning Research**, [s. l.], 2026. Disponível em: https://openreview.net/forum?id=o4pXVP8RCD. Acesso em: 22 out. 2026.

LEOTTA, M. et al. Capture-replay vs. programmable web testing: an empirical assessment during test case evolution. In: WORKING CONFERENCE ON REVERSE ENGINEERING, 20., 2013, Koblenz. **Anais** [...]. Piscataway: IEEE, 2013. p. 272-281. DOI: 10.1109/WCRE.2013.6671302.

LEOTTA, M. et al. Visual vs. DOM-based web locators: an empirical study. In: INTERNATIONAL CONFERENCE ON WEB ENGINEERING, 14., 2014, Toulouse. **Anais** [...]. Cham: Springer, 2014. p. 322-340. DOI: 10.1007/978-3-319-08245-5_19.

LEOTTA, M. et al. Robula+: an algorithm for generating robust XPath locators for web testing. **Journal of Software: Evolution and Process**, [s. l.], v. 28, n. 3, p. 177-204, mar. 2016. DOI: 10.1002/smr.1771.

MICROSOFT. **Locators**: Playwright Python. [S. l.], [2026]. Disponível em: https://playwright.dev/python/docs/locators. Acesso em: 16 set. 2026.

NASS, M. et al. Similarity-based web element localization for robust test automation. **ACM Transactions on Software Engineering and Methodology**, New York, v. 32, n. 3, art. 75, p. 1-30, abr. 2023. DOI: 10.1145/3571855.

PRŮCHA, P.; MATOUŠKOVÁ, M.; STRNAD, J. Are LLM agents the new RPA? A comparative benchmark study of agentic automation and RPA across standardized workflow tasks. **Cogent Business & Management**, [s. l.], v. 13, n. 1, art. 2724760, 2026. DOI: 10.1080/23311975.2026.2724760.

QI, X.; QIAN, X.; LI, Y. Semantic test repair for web applications. In: ACM JOINT EUROPEAN SOFTWARE ENGINEERING CONFERENCE AND SYMPOSIUM ON THE FOUNDATIONS OF SOFTWARE ENGINEERING, 31., 2023, San Francisco. **Anais** [...]. New York: ACM, 2023. p. 1190-1202. DOI: 10.1145/3611643.3616324.

STOCCO, A.; YANDRAPALLY, R.; MESBAH, A. Visual web test repair. In: ACM JOINT EUROPEAN SOFTWARE ENGINEERING CONFERENCE AND SYMPOSIUM ON THE FOUNDATIONS OF SOFTWARE ENGINEERING, 26., 2018, Lake Buena Vista. **Anais** [...]. New York: ACM, 2018. p. 503-514. DOI: 10.1145/3236024.3236063.

VAN DER AALST, W. M. P.; BICHLER, M.; HEINZL, A. Robotic process automation. **Business & Information Systems Engineering**, [s. l.], v. 60, n. 4, p. 269-272, ago. 2018. DOI: 10.1007/s12599-018-0542-4.

XUE, T. et al. An illusion of progress? Assessing the current state of web agents. In: CONFERENCE ON LANGUAGE MODELING, 2., 2025, Montreal. **Anais** [...]. [S. l.: s. n.], 2025. Disponível em: https://openreview.net/forum?id=6jZi4HSs6o. Acesso em: 16 set. 2026.